

Лабораторная работа № 9.

Обновление и сопровождение REST API в Production

Тема: Обновление версии: migrations, restart/redeploy, health check, smoke tests и анализ logs

Цель: Выполнить учебное production-обновление Internet Shop REST API и подтвердить его работоспособность через health checks, smoke tests и logs.

Дисциплина: Разработка браузерных приложений

Лабораторная работа №9. Обновление и сопровождение REST API в Production

В лабораторной работе необходимо выполнить учебное обновление Internet Shop REST API в production-подобной среде: подготовить новую версию, сделать backup PostgreSQL, применить обновление, выполнить health checks и smoke tests, проанализировать logs и описать rollback plan.

1. Ожидаемый результат

- Подготовлена новая версия Internet Shop REST API.
- Перед обновлением выполнена проверка tests или CI.
- Создан backup PostgreSQL перед изменениями.
- Новая версия получена на сервере или опубликована на Render.
- Выполнены npm ci, prisma generate и prisma migrate deploy, если используется Linux-сервер.
- Node.js service перезапущен или Render deployment завершён успешно.
- Проверены /health/live, /health/ready и ключевые REST endpoints.
- Проанализированы application logs.
- Описан rollback plan.
- Сформирован production update checklist.

2. Исходные условия

- Проект Internet Shop REST API уже опубликован на Linux-сервере или Render.
- Есть Git repository с актуальным кодом.
- Есть PostgreSQL database.
- Есть health endpoints /health/live и /health/ready.
- Есть доступ к logs: journalctl/Nginx logs или Render logs.
- Есть возможность выполнить smoke tests через curl, PowerShell или Postman.

3. Подготовка безопасного изменения

Внесите небольшое безопасное изменение, которое легко проверить после deployment. Например, добавьте или обновите version endpoint.

Пример version endpoint

```
app.get('/version', (req, res) => {
  res.json({
    name: 'internet-shop-api',
    version: process.env.APP_VERSION ?? 'dev'
  });
});
```

Endpoint не должен возвращать secrets, DATABASE_URL, JWT_SECRET или внутренние пути сервера.

4. Commit новой версии

```
Git

git status
git add .
git commit -m "Add API version endpoint"
git push
```

Запишите commit hash новой версии. Он понадобится для отчёта и rollback plan.

```
Commit hash

git rev-parse --short HEAD
```

5. Проверка перед deployment

```
Локальная проверка

npm ci
npx prisma generate
npm test
```

Если в проекте настроен GitHub Actions, дождитесь успешного CI run. Нельзя обновлять production, если тесты уже падают до deployment.

6. Backup PostgreSQL перед обновлением

Если новая версия затрагивает database schema или данные, перед обновлением сделайте backup. Даже если migration в этой лабораторной отсутствует, выполните backup как тренировку production-процесса.

```
Backup на Linux-сервере

sudo -u postgres pg_dump shop_db > /tmp/shop_db_$(date +%Y%m%d_%H%M%S).sql
```

```
Проверка backup

ls -lh /tmp/shop_db_*.sql
```

Backup содержит данные. Не прикладывайте полный dump к отчёту и не отправляйте его в Git.

7. Вариант А: обновление Linux deployment

Если ваше приложение развёрнуто на Ubuntu Server, выполните обновление на сервере.

```
Получение новой версии

ssh ubuntu@SERVER_IP
sudo -iu shop
```

```
cd /var/www/internet-shop
git fetch origin
git pull --ff-only origin main
```

Обновление dependencies и Prisma

```
npm ci
set -a
source /etc/internet-shop.env
set +a
npx prisma generate
npx prisma migrate deploy
```

Restart service

```
exit
sudo systemctl restart internet-shop
sudo systemctl status internet-shop --no-pager
```

8. Вариант B: обновление Render deployment

Если приложение опубликовано на Render, push в deployment branch должен запустить новый deployment или подготовить его к ручному запуску.

1. Откройте Render Dashboard.
2. Откройте Web Service Internet Shop API.
3. Перейдите в Deploys.
4. Найдите новый deployment.
5. Откройте deploy logs.
6. Проверьте build command.
7. Проверьте prisma generate.
8. Проверьте migrate deploy, если он выполняется в start command.
9. Дождитесь успешного health check.

9. Health checks после обновления

Linux-сервер

```
curl http://127.0.0.1:3000/health/live
curl http://127.0.0.1:3000/health/ready
curl http://SERVER_IP/health/ready
```

Render или HTTPS API

```
curl https://YOUR_API_URL/health/live
curl https://YOUR_API_URL/health/ready
```

10. Smoke tests

Smoke tests должны быстро проверить самые важные сценарии API. Они не заменяют весь набор Jest/Supertest-тестов.

Минимальный smoke test

API=http://SERVER_IP

```
curl -i $API/version
curl -i $API/api/products
curl -i $API/health/ready
```

Для Render замените API на публичный HTTPS URL сервиса.

11. Проверка authentication

Проверьте login и один protected endpoint. Не публикуйте реальные пароли или токены в отчёте.

Схема проверки

```
POST /api/auth/login
↓
accessToken
↓
Authorization: Bearer <token>
↓
protected endpoint
```

12. Анализ logs на Linux

Application logs

```
sudo journalctl -u internet-shop -n 200 --no-pager
```

Nginx logs

```
sudo tail -n 100 /var/log/nginx/access.log
sudo tail -n 100 /var/log/nginx/error.log
```

13. Анализ logs на Render

10. Откройте Web Service.
11. Перейдите в Logs.
12. Найдите startup logs новой версии.
13. Проверьте ошибки migration/startup.
14. Выполните smoke test.
15. Найдите соответствующие runtime logs.

14. Фиксация успешного release

Если health checks, smoke tests и logs не показывают ошибок, release можно считать успешным.

Release record

Поле	Значение
Дата и время	Заполнить

Поле	Значение
Версия или commit	Заполнить
Среда	Linux VPS или Render
Backup создан	Да/нет
Migrations применены	Да/нет
Health checks	Passed/failed
Smoke tests	Passed/failed
Ошибки в logs	Нет/описать

15. Симуляция ошибки

Для учебной проверки можно смоделировать безопасную ошибку только в тестовой или учебной среде. Например, временно сделать неверный APP_VERSION или вернуть ошибочный status в неключевом endpoint. Не ломайте production database.

16. Внесите безопасное ошибочное изменение.
17. Опубликуйте его в учебной среде.
18. Найдите проблему через health/smoke/logs.
19. Опишите, где именно ошибка обнаружилась.
20. Верните корректную версию.

16. Rollback plan для Linux

Rollback приложения выполняется только после понимания, затрагивала ли новая версия database schema.

Rollback кода

```
sudo -iu shop
cd /var/www/internet-shop
git log --oneline -10
git checkout <PREVIOUS_GOOD_COMMIT>
npm ci
set -a
source /etc/internet-shop.env
set +a
npx prisma generate
exit
sudo systemctl restart internet-shop
```

Если migration изменила database schema, rollback приложения может быть недостаточен. В отчёте обязательно укажите, менялась ли database.

17. Rollback plan для Render

21. Откройте Render Dashboard.
22. Перейдите в Deploys.
23. Найдите предыдущий successful deployment.
24. Запустите redeploy предыдущей версии, если это доступно.
25. Проверьте /health/ready.
26. Выполните smoke tests.

27. Проверьте logs.

18. Production update checklist

- 28. Проверить git status.
- 29. Зафиксировать commit или tag.
- 30. Проверить CI/tests.
- 31. Проверить список environment changes.
- 32. Сделать backup database.
- 33. Получить новую версию на сервере или запустить Render deploy.
- 34. Выполнить npm ci.
- 35. Выполнить prisma generate.
- 36. Выполнить prisma migrate deploy.
- 37. Перезапустить service или дождаться redeploy.
- 38. Проверить health endpoints.
- 39. Выполнить smoke tests.
- 40. Проверить logs.
- 41. Зафиксировать результат release.
- 42. Подготовить rollback notes.

19. Контрольные вопросы

- 43. Чем deployment отличается от release?
- 44. Зачем фиксировать commit hash?
- 45. Почему перед migration нужен backup?
- 46. Почему database rollback сложнее rollback кода?
- 47. Какая команда применяет Prisma migrations в production?
- 48. Почему после deployment нужны smoke tests?
- 49. Где смотреть logs на Linux?
- 50. Где смотреть logs на Render?
- 51. Что делать при 502 Bad Gateway после обновления?
- 52. Какие проверки входят в минимальный production checklist?

20. Требования к отчёту

- 53. Цель лабораторной работы.
- 54. Commit hash новой версии.
- 55. Скриншот успешного CI или локальных tests.
- 56. Команда и результат backup PostgreSQL.
- 57. Команды обновления Linux deployment или скриншоты Render deployment.
- 58. Результат /health/live.
- 59. Результат /health/ready.
- 60. Результат GET /api/products.
- 61. Результат /version.
- 62. Фрагмент logs после обновления.
- 63. Описание smoke tests.
- 64. Rollback plan.
- 65. Production update checklist.
- 66. Ответы на контрольные вопросы.

21. Критерии выполнения

Критерии

Критерий	Результат
Новая версия подготовлена и закоммичена	Обязательно

Критерий	Результат
Перед обновлением выполнены tests/CI	Обязательно
Backup PostgreSQL создан	Обязательно
Deployment выполнен	Обязательно
Migrations применены или указано, что migrations отсутствуют	Обязательно
Health checks проходят	Обязательно
Smoke tests проходят	Обязательно
Logs проверены	Обязательно
Rollback plan описан	Обязательно
Production checklist составлен	Обязательно

22. Вывод

В результате лабораторной работы студент выполняет полный цикл сопровождения production REST API: готовит новую версию, проверяет её, создаёт backup, обновляет приложение, применяет migrations, проверяет health endpoints и smoke tests, анализирует logs и описывает rollback plan.